

REST + RabbitMQ + Redis + NGINX

Sistema Escalable d'Adquisició d'Entrades de Concert

Informe Tècnic

Biel Gregori Piñas

Efren Costas Sánchez

Òscar Fortuny Velasco

Universitat Rovira i Virgili - Sistemes Distribuïts

10-4-2026

https://github.com/bielgregori/SD_Task1

Índex

Contenido

1. Introducció	3
2. Descripció del Sistema	4
2.1 Visio General	4
2.2 Models d'Entrades	4
2.2.1 Entrades Sense Numero	4
2.2.2 Entrades Numerades	4
2.3 Arquitectures de Comunicació	4
2.3.1 Arquitectura de Comunicació Directa	4
2.3.2 Arquitectura de Comunicacio Indirecta (RabbitMQ)	5
3. Benchmark i Carregues de Treball	6
3.1 Fitxers de Benchmark	6
3.2 Verificació de Correctesa	6
4. Comparació Arquitectònica	7
4.1 Comunicació Directa vs. Indirecta	7
4.2 Estratègia de Balanceig de Carrega	7
4.3 Mecanismes de Consistència	7
4.3.1 Entrades Sense Numero	7
4.3.2 Entrades Numerades	7
4.4 Colls d'Ampolla i Limitacions	8
5. Resultats Experimentals	9
5.1 Configuracio Experimental	9
5.2 Latència	9
5.2 Throughput	11
5.3 Escalabilitat	12
6. Discussió Conceptual	13
6.1 Compromís entre Throughput i Consistència	13
6.2 Impacte de la Contenció en el Rendiment	13
6.3 Adequació per a Sistemes del Mon Real	13
7. Conclusions	14

1. Introducció

Aquest informe presenta el disseny, la implementació i l'avaluació experimental d'un sistema distribuït escalable de venda d'entrades, desenvolupat com a pràctica de l'assignatura de Sistemes Distribuïts. El sistema simula una plataforma real de venda d'entrades per a concerts, amb 20.000 seients disponibles.

La pràctica requereix implementar i comparar dos paradigmes de comunicació fonamentalment diferents:

- Comunicació directa (REST)
- Comunicació indirecta mitjançant cues de missatges (RabbitMQ)

S'han implementat dos models d'entrades, les no numerades (s'han de vendre com a màxim 20000 entrades les quals son totes idèntiques), i les numerades (s'han de vendre també 20000 entrades però cada entrada és única i per tant només s'ha de vendre un sol cop).

Els experiments de la pràctica s'han executat tant en local com en varies màquines per comprovar el funcionament correcte en un sistema distribuït.

2. Descripció del Sistema

2.1 Visio General

El sistema esta compost per tres capes lògiques principals:

1. Capa de clients: genera i envia sol·licituds de compra d'entrades basant-se en els fitxers de benchmark proporcionats.
2. Capa de middleware / comunicació: encamina les peticions cap als workers utilitzant comunicació directa o indirecta.
3. Capa de workers: processa les peticions, aplica les restriccions de correctesa i persisteix l'estat en el backend de consistència compartit.

2.2 Models d'Entrades

2.2.1 Entrades Sense Numero

En el model d'entrades sense numero, totes les entrades son intercanviables. El sistema imposa un límit global de exactament 20.000 compres . Totes les sol·licituds posteriors son rebutjades amb una resposta d'error adequada.

2.2.2 Entrades Numerades

En el model d'entrades numerades, les localitats estan identificades de l'1 al 20.000. Cada localitat nomes es pot vendre una vegada, i el sistema ha de gestionar correctament els intents concurrents sobre la mateixa localitat. Aquest model estressa els mecanismes de consistència del backend.

Per assegurar aquestes propietat hem utilitzat el backend de consistència de comptador atòmic de Redis

2.3 Arquitectures de Comunicació

2.3.1 Arquitectura de Comunicació Directa

L'arquitectura directa implementa comunicació síncron de petició-resposta entre clients i servidors.

Middleware utilitzat: REST (HTTP)

Els clients envien peticions de compra a un únic punt d'entrada. La carrega es distribueix entre diverses instancies de workers utilitzant l'estrategia següent:

Estrategia de balanceig de carrega: Proxy invers NGINX (round-robin cap als workers REST)

2.3.2 Arquitectura de Comunicacio Indirecta (RabbitMQ)

L'arquitectura indirecta desacobla els clients dels workers mitjançant RabbitMQ com a broker de missatges. Els clients publiquen peticions de compra a una cua; els workers consumeixen i processen els missatges de manera asíncrona.

Propietats clau d'aquesta arquitectura:

- Els clients estan desacoblats de la disponibilitat dels workers
- Els workers es poden escalar independentment
- La durabilitat dels missatges es pot configurar per a tolerancia a fallades

2.4 Backend de Consistencia

Per garantir que les entrades no es venguin de mes i que les localitats numerades no es venguin dues vegades, tots els workers comparteixen un backend de consistencia.

Backend escollit per a aquesta implementacio: Redis — **Lua script (GET + INCR condicional)** per a entrades sense número, **SADD sobre un Set** per a entrades numerades.

3. Benchmark i Carregues de Treball

3.1 Fitxers de Benchmark

S'han utilitzat dos fitxers de benchmark per dirigir l'avaluació:

Fitxer	Format	Model
benchmark_unnumbered.txt	BUY <client_id> <request_id>	Sense numero
benchmark_numbered.txt	BUY <client_id> <seat_id> <request_id>	Numerada

3.2 Verificació de Correctesa

Després de cada execució del benchmark, s'ha verificat la correctesa comprovant:

4. El nombre total d'operacions BUY reeixides es exactament 20.000 (sense numero) o com a màxim una per localitat (numerada)
5. No hi ha assignacions de localitat duplicades en el model numerat
6. Totes les peticions rebutjades han rebut la resposta d'error adequada

4. Comparació Arquitectònica

4.1 Comunicació Directa vs. Indirecta

Les dues architectures representen filosofies de comunicació fonamentalment diferents:

Dimensió	Directa (REST/RPC)	Indirecta (RabbitMQ)
Acoblament	Fort: client bloquejat fins a resposta	Feble: client envia i oblida
Latència	Baixa (round-trip síncron)	Mes alta (retard de processament a la cua)
Throughput	Limitat per la capacitat del servidor	Desacoblat per la cua de missatges
Escalabilitat	Requereix balancejador de carrega	Natural: afegir/eliminar workers
Tolerancia a fallades	Falla si el Load Balancer cau (SPOF). Poden fallar workers i ser afegits	La cua absorbeix les peticions. Falla si cau el servidor RabbitMQ (SPOF). Poden fallar workers i ser afegits
Observabilitat	Simple (codis d'estat HTTP)	Requereix monitorització de la cua
Complexitat	Menor	Major (gestió del broker)

4.2 Estratègia de Balanceig de Carrega

A l'arquitectura directa, el balanceig de carrega s'ha implementat mitjançant NGINX. Els clients envien totes les peticions a un únic endpoint (NGINX), que les distribueix entre els workers REST en mode round-robin. Això permet afegir o eliminar workers simplement modificant el bloc upstream de la configuració d'NGINX.

A l'arquitectura indirecta, el balanceig de carrega es inherent al model de missatgeria. RabbitMQ distribueix els missatges entre els consumidors amb un repartiment round-robin per defecte, amb límits de prefetch que controlen la concurrència per worker.

4.3 Mecanismes de Consistència

4.3.1 Entrades Sense Numero

El comptador global s'ha implementat mitjançant un **script Lua** a Redis que executa atòmicament GET + INCR condicional sobre la clau tickets:unnumbered:sold. Si el valor actual és menor que 20.000, s'incrementa i es confirma la compra; en cas contrari, es retorna rebuig **sense modificar el comptador**, evitant qualsevol necessitat de revertir.

4.3.2 Entrades Numerades

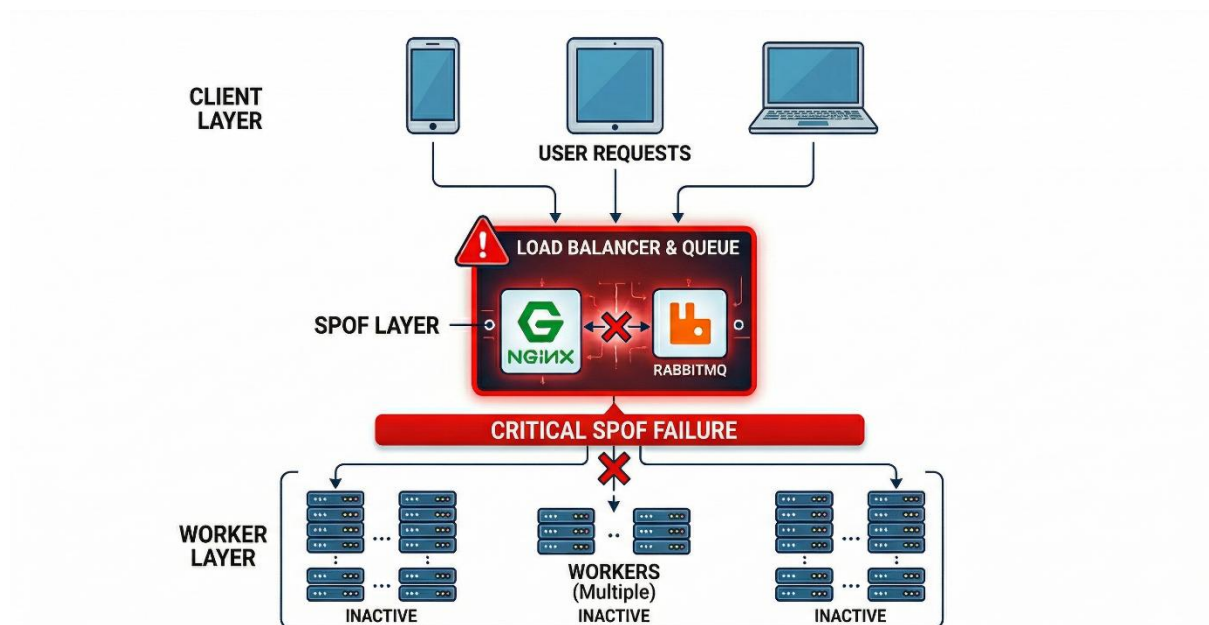
Per a les entrades numerades s'utilitza **SADD** sobre un Set Redis amb clau tickets:numbered:seats. SADD és atòmic i retorna 1 si l'asiento s'ha afegit (no existia)

o 0 si ja estava venut. Això garanteix que mai dos workers puguin assignar el mateix asiento simultàniament, sense necessitar claus individuals per asiento.

4.4 Colls d'Ampolla i Limitacions

Com és evident, tant en la comunicació directa i indirecta els colls d'ampolla quan hi ha poc workers són ells, ja que no tenen la capacitat de processar totes les peticions de manera més ràpida que el load balancer, la cua de missatges o bé el backend redis.

S'ha de comentar també que si cauen el servidor NGINX o la cua RabbitMQ al ser SPOF el sistema deixaria de funcionar ja que els clients estan desacoplats dels workers i no hi tenen accés sense passar per ells.



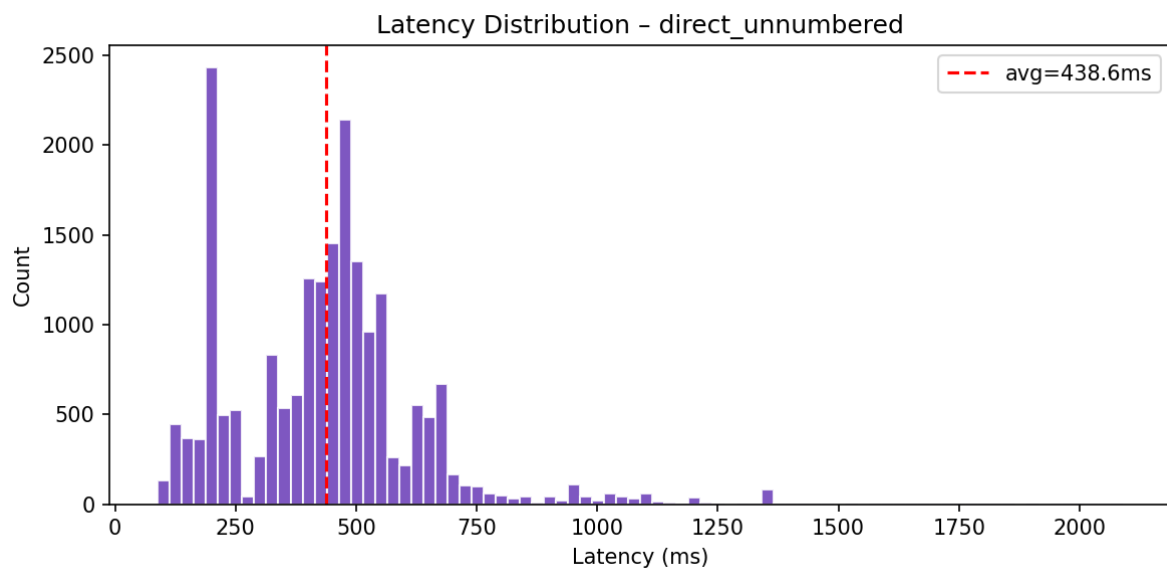
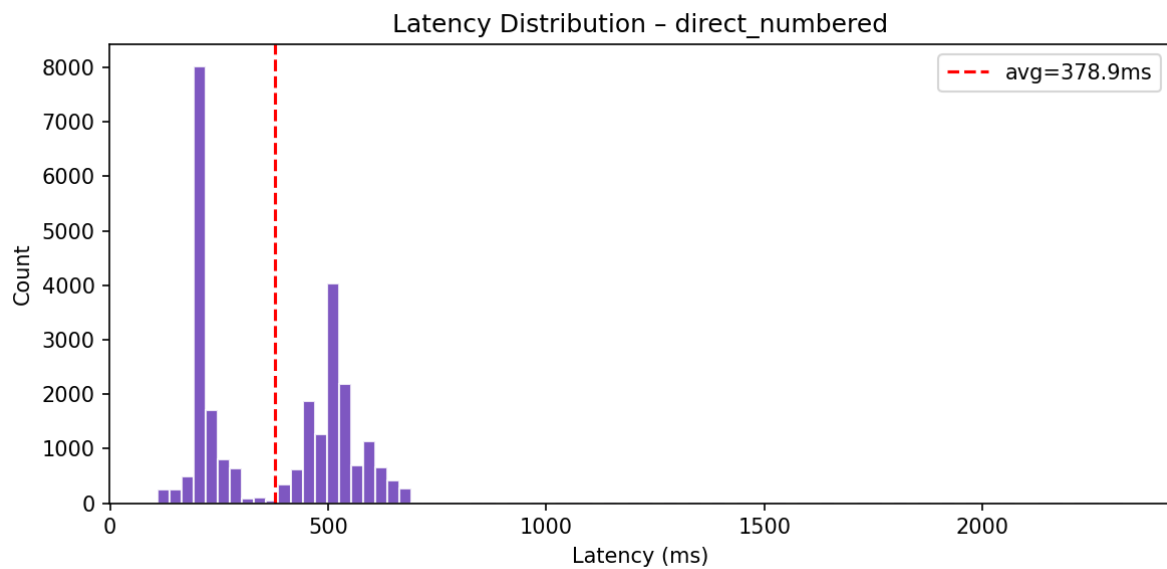
5. Resultats Experimentals

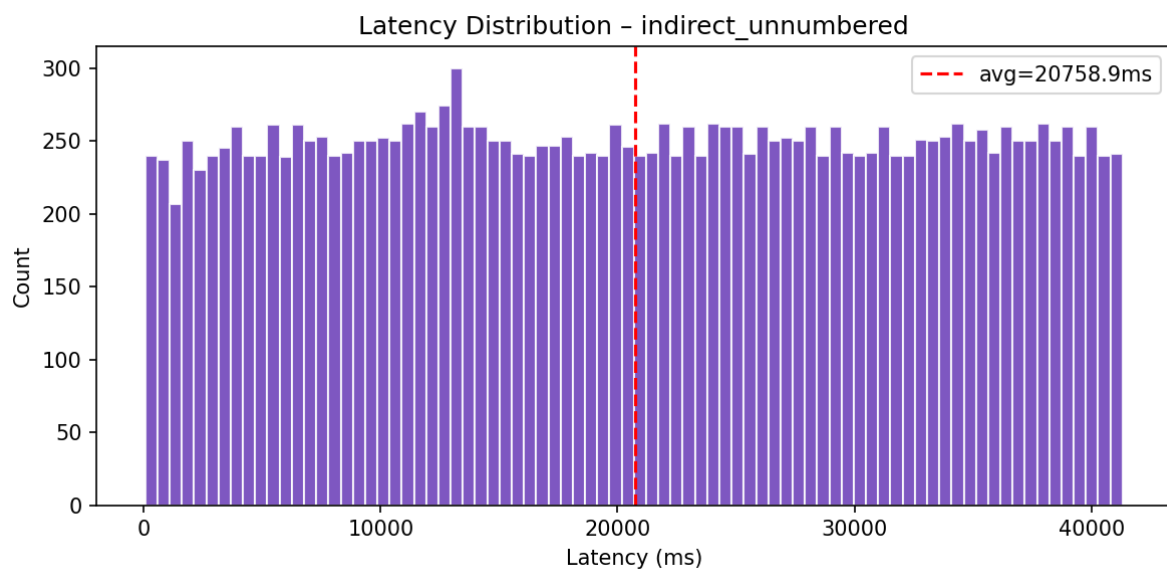
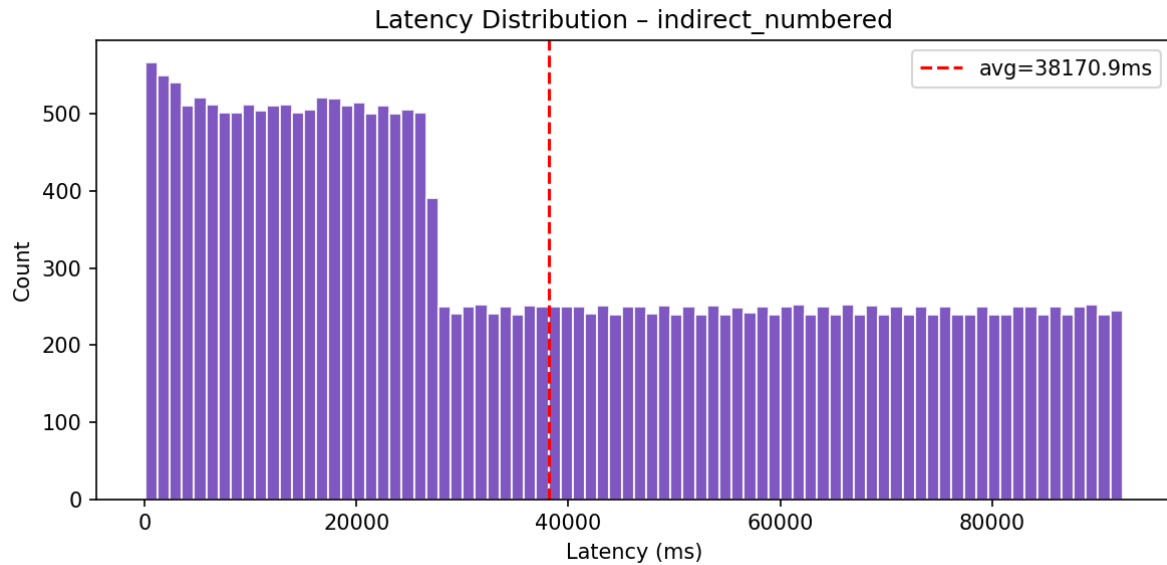
5.1 Configuracio Experimental

Les proves experimentals estan fetes des dels ordinadors del laboratori distribuint el client en una maquina, el load balancer o el rabbitmq en una altra junt amb el backend redis i els workers en una altra màquina.

5.2 Latència

Fent proves hem obtingut les diferents gràfiques de latència:

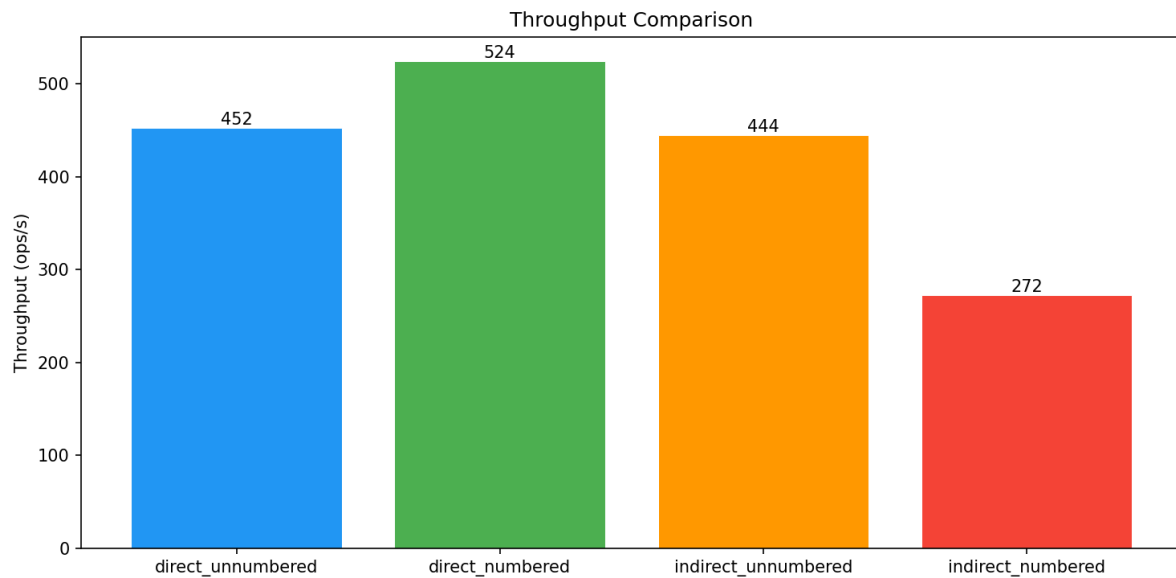




Podem observar que la latència de les entrades quan es fa servir una comunicació directa es molt menor ja que no passen per una cua en la que esperen ser ateses.

Pel contrari, en la comunicació indirecta, les peticions son retingudes en una cua de missatges en la que esperen fins a ser processades per els workers el qual incrementa notablement el temps de resposta quan s'envien moltes peticions simultàniament.

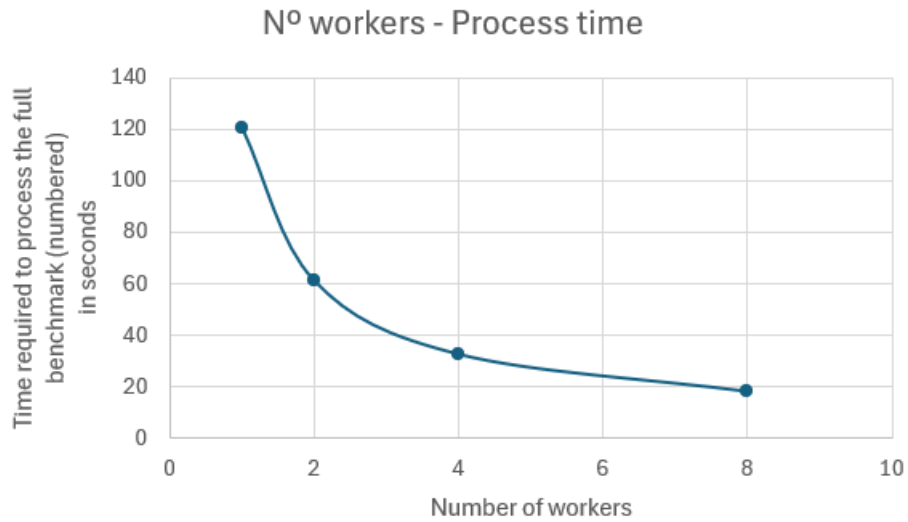
5.2 Throughput



En aquesta gràfica veiem la velocitat de processament de cada sistema utilitzant els dos tipus d'entrades, podem observar un fenomen: Com sembla lògic, en la indirecta podem veure que el throughput més alt és amb les entrades sense numerar, conclusió a la que arribaríem al pensar que les comprovacions que s'han de fer per vendre la entrada són menors, la qual es correcte en aquest cas. Tot i així, si apliquem la mateixa lògica en la comunicació directa veiem que no es compleix, això es deu a que en les entrades numerades tots els workers competeixen pel contador d'entrades, en canvi en les numerades, cada worker pot treballar amb una entrada diferent sense molestar-se.

5.3 Escalabilitat

En quant a la escalabilitat ens vam centrar en veure el temps en el que el sistema podia processar totes les peticions del benchmark depenent del nombre de workers que hi haguessin funcionant per veure si la millora era notable:



Podem veure que l'increment de workers disminueix notablement el temps de processament, tot i així la gràfica no es lineal, el que ens assegura que en un cert punt de workers, l'increment d'aquests pràcticament no disminuirà el temps de processament i per tant s'hauria de tractar el coll d'ampolla que està retardant el sistema (load balancer, rabbitmq, redis...).

6. Discussió Conceptual

6.1 Compromís entre Throughput i Consistència

La tensió fonamental en els sistemes distribuïts de venda d'entrades es entre maximitzar el throughput i garantir la consistència. Aquests dos objectius tiren en direccions oposades:

La consistència forta (transaccions serialitzables, bloquejos distribuïts) garanteix la correctesa però crea colls d'ampolla de serialització. Si vulguessim afegir més consistència amb més verificacions el que fariem es augmentar el temps de procés i conseqüentment el throughput.

Tot i això creiem que el nostre projecte mostra perfectament una alta consistència junt amb un throughput alt que permeten al sistema ser una aplicació completament viable en el món real.

6.2 Impacte de la Contenció en el Rendiment

Els nostres experiments confirmen que la contenció es el limitador de rendiment principal en aquesta classe de carrega. S'han observat dos règims diferenciats:

1. Baixa contenció (entrades numerades, distribució uniforme): escalat gairebé lineal amb els workers. Cada worker gestiona localitats independents; el temps de bloqueig al backend es negligible.
2. Contenció mitjana (entrades sense numero): escalat moderat. El comptador global es converteix en un punt de serialització, però les operacions d'increment atòmic son extremadament ràpides a Redis.

6.3 Adequació per a Sistemes del Món Real

Basant-nos en la implementació del sistema creiem que per al món real la millor opció és utilitzar la comunicació indirecta ja que el desacoblament temporal aporta molts punts a favor d'aquesta els quals contraresten la dificultat d'implementació (la qual no es notablement més elevada que en l'altre cas). A més la cua permet absorbir pics alts de peticions (escenari molt comú en ventes d'entrades el moment que s'obren i tothom vol comprar).

7. Conclusions

Aquest projecte ha demostrat el disseny i la implementació d'un sistema de venda d'entrades distribuït i escalable sota condicions realistes d'alta concurrència. Les principals conclusions son:

1. Ambdues arquitectures apliquen correctament els límits d'entrades i la unicitat de localitats sota carrega concurrent, quan el backend de consistència està correctament dissenyat.
2. L'arquitectura indirecta RabbitMQ proporciona una millor tolerància a fallades i a pics de peticions però introdueix major latència i complexitat operacional.
3. L'arquitectura REST directa proporciona menor latència i desplegament més simple, però requereix un ajust acurat del balancejador de carrega i del backend per escalar.